

FlowAccount Backend Challenge — สิ่งที่ต้องทำ, Validation Rules, และผลทดสอบ

เอกสารนี้อธิบายโจทย์ ระบบจัดการสินค้า (Product Management) ตาม FlowAccount Engineering Workshop (Track A: Backend) โดยอ้างอิงผลจริงจาก `scripts/result.txt`

หมายเหตุเกี่ยวกับชุดทดสอบ: เซิร์ฟเวอร์เก็บข้อมูลในหน่วยความจำ และรับ curl กับข้อมูลเดิมอยู่แล้ว ดังนั้นเคสสร้างสินค้าใหม่ในรอบนี้จึงได้ **HTTP 400 (SKU ซ้ำ)** ไม่ใช่ 201 แต่สินค้า 4 รายการยังอยู่ในระบบ (id 1-4) และเคสอื่น ๆ ครอบคลุม validation / business logic ครบ

สถานการณ์และข้อมูลพื้นฐาน

FlowAccount ต้องการระบบจัดการสินค้าอย่างง่ายสำหรับร้านค้า SME ในประเทศไทย

ฟิลด์	ความหมาย	ข้อกำหนดจากโจทย์
name	ชื่อสินค้า (Product Name)	ต้องไม่ว่าง
sku	รหัสสินค้า	ต้องไม่ซ้ำกัน
price	ราคา (บาท)	ต้องมากกว่า 0
stock	จำนวนคงเหลือ	ต้องไม่ติดลบ (≥ 0)
category	หมวดหมู่	ต้องเป็น 1 ใน 4 ค่า: อาหาร, เครื่องดื่ม, ของใช้, เสื้อผ้า

API ที่ implement แล้ว:

- POST `/api/products` — Challenge 1
- GET `/api/products` — Challenge 2
- POST `/api/products/sell` — Challenge 3
- GET `/api/products/search` — Challenge 4 (Bonus)
- PUT `/api/products/bulk-price-update` — Challenge 5 (Bonus)

สินค้าที่มีอยู่ในระบบตอนรับ `result.txt`:

id	name	sku	price	stock	category
1	ข้าวผัด	FOOD001	50 (หลังอัปเดตราคา)	14 (หลังขาย)	อาหาร
2	น้ำเปล่า	DRINK001	10	0	เครื่องดื่ม
3	สบู	HOME001	30 (หลังอัปเดตราคา)	15	ของใช้
4	เสื้อยืด	CLOTH001	199	8	เสื้อผ้า

Challenge 1: สร้าง API สำหรับเพิ่มสินค้าใหม่

Endpoint: POST /api/products

สิ่งที่ต้องทำ

- 1. รับข้อมูลสินค้าจาก request body
API อ่าน JSON: name, sku, price, stock, category
- 2. Validate ข้อมูลตามเงื่อนไขด้านล่าง
รวมทุก error ใส่ array errors ไม่หยุดที่ข้อแรก
- 3. ถ้า validation ผ่าน → บันทึกสินค้าและส่ง response กลับ
ตามโจทย์ควรได้ HTTP 201 Created พร้อม id และ createdAt
- 4. ถ้า validation ไม่ผ่าน → ส่ง error messages กลับไป
HTTP 400 Bad Request ในรูปแบบ { "errors": ["...", "..."] }

Validation Rules

ฟิลด์	กติกาจากโจทย์	พฤติกรรมที่ implement
Name	ต้องไม่ว่าง	ว่าง, null, หรือมีแต่ช่องว่าง → "ชื่อสินค้าต้องไม่ว่าง"
SKU	ต้องไม่ว่าง	ว่าง → "รหัสสินค้าต้องไม่ว่าง"
SKU	ยาวอย่างน้อย 3 ตัวอักษร	สั้นกว่า 3 → "รหัสสินค้าต้องมีอย่างน้อย 3 ตัวอักษร"
SKU	ต้องไม่ซ้ำกับสินค้าที่มีอยู่แล้ว	เทียบแบบไม่สนตัวพิมพ์ใหญ่-เล็ก → "รหัสสินค้าต้องไม่ซ้ำกับสินค้าที่มีอยู่แล้ว"
Price	ต้องมากกว่า 0	0, ติดลบ, หรือไม่ใช่ตัวเลข → "ราคาต้องมากกว่า 0"
Stock	ต้องไม่ติดลบ (>= 0)	ติดลบ หรือไม่ใช่ตัวเลข → "จำนวนคงเหลือต้องไม่ติดลบ"
Category	ต้องเป็น 1 ใน 4 หมวดที่กำหนด	นอกชุด → "หมวดหมู่ต้องเป็นหนึ่งใน: อาหาร, เครื่องดื่ม, ของใช้, เสื้อผ้า"

ตัวอย่าง request ตามโจทย์:

```
{
  "name": "ข้าวผัด",
  "sku": "FOOD001",
  "price": 45.00,
  "stock": 20,
  "category": "อาหาร"
}
```

ตัวอย่าง success ตามโจทย์: HTTP 201 พร้อม id, ข้อมูลสินค้า, createdAt

ตัวอย่าง error ตามโจทย์: **HTTP 400** { "errors": ["ชื่อสินค้าต้องไม่ว่าง", "รหัสสินค้าต้องมีอย่างน้อย 3 ตัวอักษร", "ราคาต้องมากกว่า 0"] }

ผลทดสอบจาก result.txt

1) ฟิลด์หายทั้งหมด — ตรวจสอบทุกกติกาพร้อมกัน

Request: `POST /api/products` body `{}`

Response **HTTP 400**:

```
{
  "errors": [
    "ชื่อสินค้าต้องไม่ว่าง",
    "รหัสสินค้าต้องไม่ว่าง",
    "ราคาต้องมากกว่า 0",
    "จำนวนคงเหลือต้องไม่ติดลบ",
    "หมวดหมู่ต้องเป็นหนึ่งใน: อาหาร, เครื่องดื่ม, ของใช้, เสื้อผ้า"
  ]
}
```

ตรงกับสิ่งที่ต้องทำข้อ 4: ส่ง error หลายข้อความกลับไปในครั้งเดียว

2) Name ต้องไม่ว่าง (รวมช่องว่างอย่างเดียว)

Request: `"name": " "` (ที่เลือกถูกต้อง)

HTTP 400: { "errors": ["ชื่อสินค้าต้องไม่ว่าง"] }

3) SKU ต้องไม่ว่าง

Request: `"sku": ""`

HTTP 400: { "errors": ["รหัสสินค้าต้องไม่ว่าง"] }

4) SKU ยาวอย่างน้อย 3 ตัวอักษร

Request: `"sku": "AB"`

HTTP 400: { "errors": ["รหัสสินค้าต้องมีอย่างน้อย 3 ตัวอักษร"] }

ตรงกับตัวอย่าง error ในโจทย์

5) Price ต้องมากกว่า 0

- `"price": 0` → **HTTP 400** "ราคาต้องมากกว่า 0"
- `"price": "abc"` → **HTTP 400** "ราคาต้องมากกว่า 0"

6) Stock ต้องไม่ติดลบ

- `"stock": -1` → **HTTP 400** "จำนวนคงเหลือต้องไม่ติดลบ"
- `"stock": "many"` → **HTTP 400** "จำนวนคงเหลือต้องไม่ติดลบ"

7) Category ต้องเป็น 1 ใน 4 หมวด

Request: "category": "ของเล่น"

HTTP 400: "หมวดหมู่ต้องเป็นหนึ่งใน: อาหาร, เครื่องดื่ม, ของใช้, เสื้อผ้า"

8) คลาย error พร้อมกัน (ตามตัวอย่างโจทย์)

Request: name ว้าง, sku = AB, price = 0, stock = -1, category = ของเล่น

HTTP 400:

```
{
  "errors": [
    "ชื่อสินค้าต้องไม่ว่าง",
    "รหัสสินค้าต้องมีอย่างน้อย 3 ตัวอักษร",
    "ราคาต้องมากกว่า 0",
    "จำนวนคงเหลือต้องไม่ติดลบ",
    "หมวดหมู่ต้องเป็นหนึ่งใน: อาหาร, เครื่องดื่ม, ของใช้, เสื้อผ้า"
  ]
}
```

9) SKU ต้องไม่ซ้ำ

พยายามสร้าง FOOD001, DRINK001, HOME001, CLOTH001 และ food001 อีกครั้งบนเซิร์ฟเวอร์ที่มีข้อมูลอยู่แล้ว

ทุกเคสได้ **HTTP 400:** "รหัสสินค้าต้องไม่ซ้ำกับสินค้าที่มีอยู่แล้ว"

เคส food001 ยืนยันว่า uniqueness ไม่สนใจตัวพิมพ์ใหญ่-เล็ก

สอบนี้ไม่ได้ 201 เพราะ store ไม่ว่าง — แต่รายการสินค้า id 1-4 ใน Challenge 2 ยืนยันว่าเคยสร้างสำเร็จมาก่อน (มี createdAt)

Challenge 2: สร้าง API สำหรับดึงรายการสินค้า

Endpoint: GET /api/products หรือ GET /api/products?category=อาหาร

สิ่งที่ต้องทำ

1. ดึงสินค้าทั้งหมดจาก storage

ไม่มี category (หรือส่งค่าว่าง) → คืน array ทั้งก้อน

2. ถ้ามี query parameter category → filter เฉพาะสินค้าในหมวดนั้น

คืนเฉพาะแถวที่ category ตรงกับทุกตัวอักษร

3. ส่ง list ของสินค้ากลับ

เป็น JSON array

โจทยไม่ได้กำหนด validation ของ `category` ตอน list แต่ implementation ปฏิเสธหมวดที่ไม่ใช่ 4 คำที่กำหนด เพื่อไม่ให้ filter แล้วได้ `[]` โดยที่ผู้เรียกพิมพ์หมวดผิด

Validation / กติกาเพิ่มเติมที่ใช้จริง

กติกา	ผลลัพธ์
ไม่มี <code>category</code>	200, สิ้นคำทั้งหมด
<code>category=</code> (ว่าง)	200, สิ้นคำทั้งหมด
<code>category</code> เป็น 1 ใน 4 หมวด	200, เฉพาะหมวดนั้น
<code>category</code> นอกชุด	400, error หมวดหมู่

ผลทดสอบจาก `result.txt`

1) ดึงสินค้าทั้งหมด

`GET /api/products` → **HTTP 200** ได้ 4 รายการ (ข้าวผัด, น้ำเปล่า, สบู่, เสื้อยืด)

2) `category` ว่างยังดึงทั้งหมด

`GET /api/products?category=` → **HTTP 200** ชุดเดียวกัน 4 รายการ

3) Filter หมวด อาหาร

HTTP 200: เหลือเพียง id 1 ข้าวผัด / FOOD001

4) Filter หมวด เสื้อผ้า

HTTP 200: เหลือเพียง id 4 เสื้อยืด / CLOTH001

5) หมวดที่ไม่มีในระบบ

`category=ของเล่น` → **HTTP 400** "หมวดหมู่ต้องเป็นหนึ่งใน: อาหาร, เครื่องดื่ม, ของใช้, เสื้อผ้า"

Challenge 3: สร้าง API สำหรับขายสินค้า (ตัดสต็อก)

Endpoint: `POST /api/products/sell`

สิ่งที่ต้องทำ

- 1. รับข้อมูล `productId` และ `quantity` ที่ต้องการขาย จาก request body
- 2. ค้นหาสินค้าจาก `productId`
- 3. ตรวจสอบว่ามีสินค้าในระบบหรือไม่
- 4. ตรวจสอบว่า stock เพียงพอหรือไม่
- 5. ถ้าผ่านทุกเงื่อนไข → ตัดสต็อก และส่ง response กลับ
 - `stock = stock - quantity`

Business Logic ที่ต้องคิด (ตรวจสอบลำดับ)

โจทย์บังคับลำดับนี้:

- 1. quantity > 0 หรือไม่
- 2. มีสินค้าในระบบหรือไม่
- 3. stock >= quantity หรือไม่
- 4. ผ่านทุกข้อ → stock = stock - quantity

ลำดับ	กติกา	HTTP	ข้อความจากระบบ
1	quantity ต้องมากกว่า 0 (ขาด, 0, ติดลบ, ไม่ใช่ตัวเลข)	400	จำนวนที่ขายต้องมากกว่า 0
2	ต้องมีสินค้าตาม productId	404	ไม่พบสินค้าในระบบ
3	stock >= quantity	400	สต็อกไม่เพียงพอ (คงเหลือ X, ต้องการขาย Y)
4	ผ่านทั้งหมด	200	ขายสินค้าสำเร็จ + สินค้าหลังตัดสต็อก

ผลทดสอบจาก result.txt

ลำดับ 1 — quantity > 0

เคส	body โดยย่อ	HTTP	errors
ไม่ส่ง quantity	{ "productId": 1 }	400	จำนวนที่ขายต้องมากกว่า 0
quantity = 0	{ "productId": 1, "quantity": 0 }	400	จำนวนที่ขายต้องมากกว่า 0
quantity ติดลบ	{ "quantity": -2 }	400	จำนวนที่ขายต้องมากกว่า 0
ไม่ใช่ตัวเลข	{ "quantity": "two" }	400	จำนวนที่ขายต้องมากกว่า 0

ยังไม่ค้นสินค้าและยังไม่ตัดสต็อก — ตรงลำดับที่โจทย์กำหนด

ลำดับ 2 — มีสินค้าในระบบหรือไม่

- { "productId": 999, "quantity": 1 } → HTTP 404 "ไม่พบสินค้าในระบบ"
- { "productId": "abc", "quantity": 1 } → HTTP 404 "ไม่พบสินค้าในระบบ"

ลำดับ 3 — stock เพียงพอหรือไม่

นำเปล่า id 2 มี stock = 0 ขาย 1 ชิ้น:

HTTP 400: "สต็อกไม่เพียงพอ (คงเหลือ 0, ต้องการขาย 1)"

ลำดับ 4 — ตัดสต็อกเมื่อผ่านทุกเงื่อนไข

{ "productId": 1, "quantity": 3 } → HTTP 200

```
{
  "message": "ขายสินค้าสำเร็จ",
  "product": {
    "id": 1,
    "name": "ข้าวผัด",
    "sku": "FOOD001",
    "price": 50,
    "stock": 14,
    "category": "อาหาร"
  },
  "soldQuantity": 3
}
```

GET /api/products หลังขายยืนยันว่าข้าวผัดเหลือ **stock 14** (ตัดจากสต็อกก่อนหน้านี้)

Challenge 4 (Bonus): ค้นหาสินค้า

Endpoint: GET /api/products/search?keyword=ข้าว

สิ่งที่ต้องทำ

1. ค้นหาสินค้าที่มี keyword ปรากฏใน name หรือ sku เป็น substring ไม่ต้องตรงทั้งคำ
2. การค้นหาต้องไม่สนใจตัวพิมพ์ใหญ่-เล็ก (case-insensitive)
food ต้องเจอ FOOD001

กติกาเสริมที่ implement: ถ้าไม่มี keyword หรือมีแต่ช่องว่าง → **HTTP 400** "กรุณาระบุ keyword"
ไม่พบรายการ → **HTTP 200** และ []

ผลทดสอบจาก result.txt

เคส	request	HTTP	ผล
ไม่ส่ง keyword	GET /api/products/search	400	กรุณาระบุ keyword
keyword เป็นช่องว่าง	keyword=	400	กรุณาระบุ keyword
ตรงใน name	keyword=ข้าว	200	ได้ข้าวผัด
ตรงใน sku แบบไม่สนใจตัวพิมพ์	keyword=food	200	ได้ FOOD001
ไม่ตรงเลย	keyword=zzzz-not-found	200	[]

เคส keyword=ข้าว ตรงตัวอย่างในโจทย์ และเคส food ยืนยัน case-insensitive uu SKU

Challenge 5 (Bonus): อัปเดตราคาเป็นชุด

Endpoint: PUT /api/products/bulk-price-update

สิ่งที่ต้องทำ

- 1. รับ array ของ { productId, newPrice }
- 2. อัปเดตราคาของแต่ละสินค้า
- 3. ส่ง summary กลับไปว่าอัปเดตสำเร็จกี่รายการ

รูปแบบ summary ที่ใช้: { "updated": N, "failed": M, "failures": [...] }
รายการที่พังไม่ทำให้รายการอื่นหยุด

Validation Rules

กติกา	HTTP	ผล
body ไม่ใช่ array	400	ต้องส่งเป็น array ของ {productId, newPrice}
array ว่าง	200	updated: 0, failed: 0
ไม่พบ productId	200 (รายการนั้น failed)	ไม่พบสินค้าในระบบ
newPrice ไม่มากกว่า 0	200 (รายการนั้น failed)	ราคาต้องมากกว่า 0
รายการถูกต้อง	นับใน updated และเขียน price ใหม่	—

กติราคา newPrice > 0 สอดคล้องกับ Challenge 1

ผลทดสอบจาก result.txt

1) ต้องเป็น array

ส่ง object { "productId": 1, "newPrice": 50 } → HTTP 400

```
{ "errors": ["ต้องส่งเป็น array ของ {productId, newPrice}"] }
```

2) array ว่าง — summary ศูนย์

HTTP 200: { "updated": 0, "failed": 0, "failures": [] }

3) ไม่พบสินค้า

[{ "productId": 999, "newPrice": 50 }] → HTTP 200 updated: 0, failed: 1, reason "ไม่พบสินค้าในระบบ"

4) ราคาไม่มากกว่า 0

- `newPrice: 0` → failed "ราคาต้องมากกว่า 0"
- `newPrice: "cheap"` → failed "ราคาต้องมากกว่า 0"

5) รายการ null ใน array

`[null]` → failed "ไม่พบสินค้าในระบบ" (`productId` เป็น `null`)

6) อัปเดตสำเร็จปนกับล้มเหลว (สรุปจำนวน)

Request:

```
[
  { "productId": 1, "newPrice": 50 },
  { "productId": 3, "newPrice": 30 },
  { "productId": 999, "newPrice": 1 },
  { "productId": 2, "newPrice": -5 }
]
```

HTTP 200:

```
{
  "updated": 2,
  "failed": 2,
  "failures": [
    { "productId": 999, "reason": "ไม่พบสินค้าในระบบ" },
    { "productId": 2, "reason": "ราคาต้องมากกว่า 0" }
  ]
}
```

ตรงสิ่งที่ต้องทำข้อ 3: สรุปว่าอัปเดตสำเร็จ **2** รายการ (id 1 ราคา 50, id 3 ราคา 30) ล้มเหลว 2 รายการ

`GET /api/products` ถ่ายไฟล์ยืนยันราคา: ข้าวผัด **50**, สลัด **30**, น้ำเปล่ายัง **10**, เสื้อมีดยง **199**

สรุปความครบของโจทย์

Challenge	สิ่งที่ต้องทำ	Validation / business rules	สถานะจาก <code>result.txt</code>
1 เพิ่มสินค้า	รับ body, validate, บันทึกหรือส่ง errors	name, sku (ว่าง / ความยาว / ซ้ำ), price > 0, stock >= 0, category 4 ค่า	400 ครบทุกกติกา; เคส 201 ไม่เกิดในรอบนี้เพราะ SKU ซ้ำกับของเดิม
2 ดึงรายการ	ดึงทั้งหมด, filter ตาม category, ส่ง list	หมวดต้องเป็น 4 ค่าถ้าส่งมา	200 ทั้งก้อน + filter อาหาร/ เสื้อผ้า; 400 หมวดผิด
3 ขาย/ตัดสต็อก	รับ id+qty, ค้นหา, เช็คว่ามีสินค้า, เช็คสต็อก, ตัดสต็อก	ตรวจสอบตามลำดับ qty → มีสินค้า → stock พอ	400 / 404 / 200 ตามลำดับ; stock ข้าวผัดเหลือ 14

Challenge	สิ่งที่ต้องทำ	Validation / business rules	สถานะจาก result.txt
4 ค้นหา	หาใน name หรือ sku, ไม่สนตัวพิมพ์	ต้องมี keyword	เจอคำว่าข้าว และ food; ไม่เจอได้ []
5 อัปเดตราคาชุด	รับ array, อัปเดตรายตัว, ส่งจำนวนที่สำเร็จ	ต้องเป็น array; ราคา > 0; มีสินค้า	updated: 2, failed: 2 ตามที่คาด